COMPUTER ENGINEERING

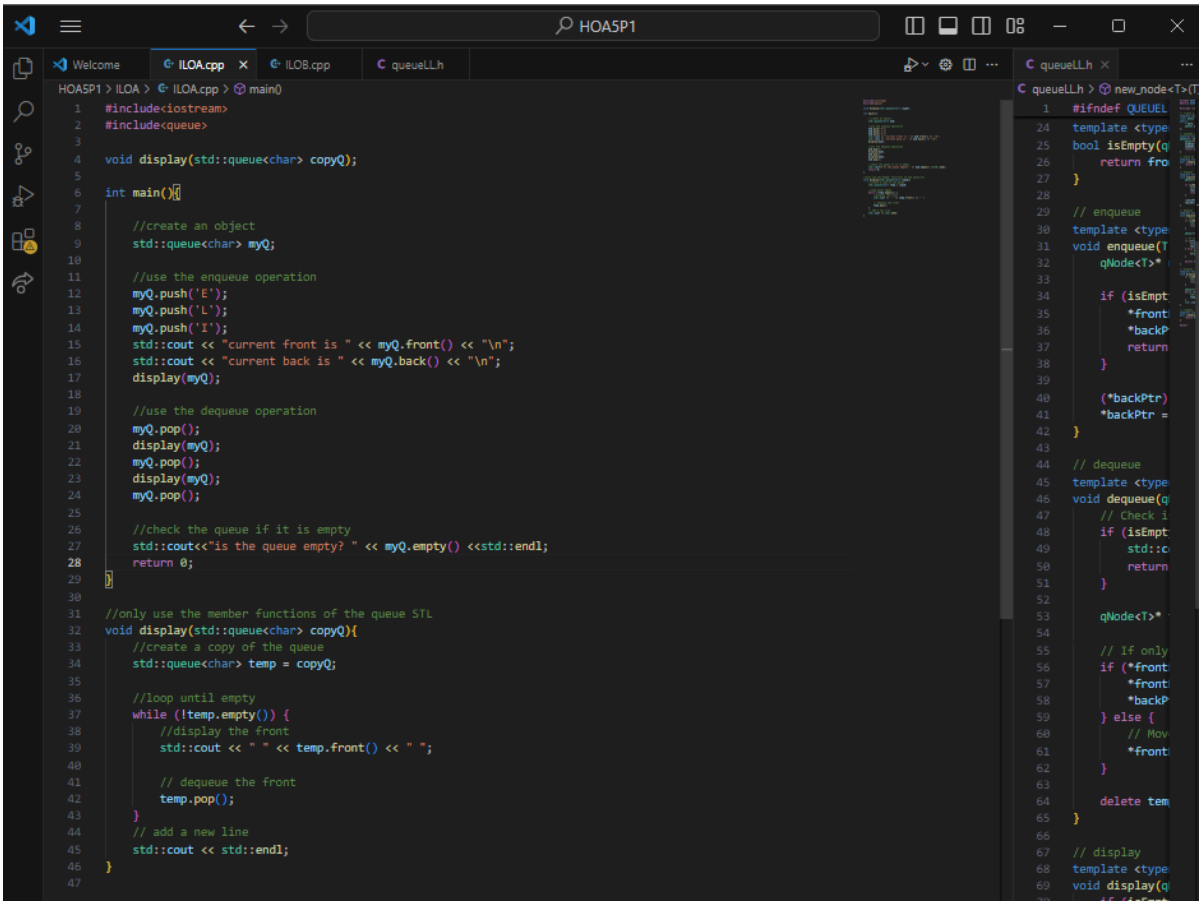
QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	08/05/26	**DATE SUBMITTED:**	08/05/26
NAME:	CRUZ, ELIJAH	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

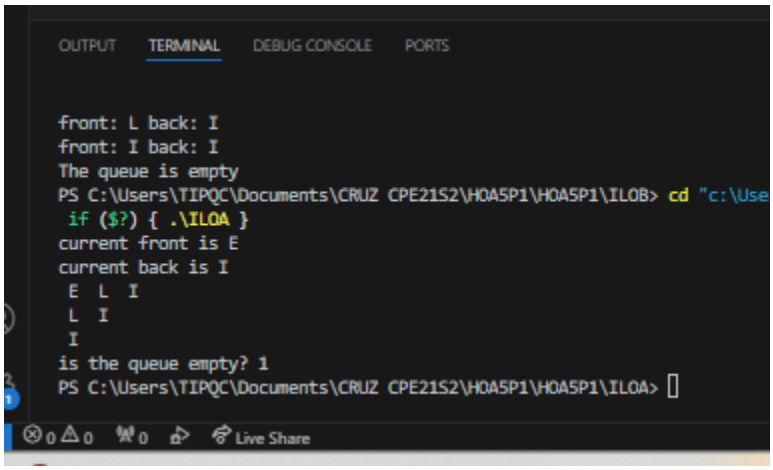
ACTIVITY NO. 5 HANDS ON ACTIVITY (PROGRESS REPORT)

1. PROCEDURE OUTPUT

ILOA



OUTPUT



OBSERVATION/REMARKS

As observed and shared during lecture, the code implements the principle of First-In, First-Out or FIFO. And utilizes a Queue data structure using singly linked lists and does operations linked to Queue's such as enqueue or dequeue. The program itself runs all the queue functions and finally outputs the three given letters while looping back to delete each character until the queue is empty once again.

ILOB

MAIN

```
#include <iostream>
#include "queueArr.h"

int main() {
   //creating an object
   queueArr<int> q(5);

   ///enqeueue 1,2,3,4
   q.Enqueue(1);

   std::cout<<"Front: " << q.Front() << std::endl;
   std::cout<<"Back: " << q.Back() << std::endl;
   std::cout<<"Size: " << q.Size() << std::endl;


   q.Enqueue(2);

   std::cout<<"Front: " << q.Front() << std::endl;
   std::cout<<"Back: " << q.Back() << std::endl;
   std::cout<<"Size: " << q.Size() << std::endl;

   q.Enqueue(3);

   std::cout<<"Front: " << q.Front() << std::endl;
   std::cout<<"Back: " << q.Back() << std::endl;
   std::cout<<"Size: " << q.Size() << std::endl;

   q.Enqueue(4);

   std::cout<<"Front: " << q.Front() << std::endl;
   std::cout<<"Back: " << q.Back() << std::endl;
   std::cout<<"Size: " << q.Size() << std::endl;
```

```
    q.Enqueue(5);

    std::cout<<"Front: " << q.Front() << std::endl;
    std::cout<<"Back: " << q.Back() << std::endl;
    std::cout<<"Size: " << q.Size() << std::endl;

    q.Enqueue(6);

    std::cout << "removed: " << q.Dequeue() << std::endl;
    std::cout << "removed: " << q.Dequeue() << std::endl;
    std::cout << "removed: " << q.Dequeue() << std::endl;

    //testing clear
    q.Clear();
    std::cout<<"Is the queue empty? " << (q.Empty() ? "yes": "no") << std::endl;

    return 0;
}
```

HEADER FILE

```
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>
#include <queue>

//array based circular queue
template <typename T>
class queueArr {
private:
    //pointer to dynamically allocate array
    T* q_array;
    //maximum number of elements a queue can hold
    size_t q_capacity;
    //current number of the elements in the queue
    size_t q_size;
    int q_front;
    //index of the back elements
    int q_back;
public:
    //constructor
    queueArr(size_t capacity = 10);

    //copy constructor
    queueArr(const queueArr& other);

    //copy assignment operator
    queueArr& operator=(const queueArr& other);

    //destructor
    ~queueArr();

    //queue operations:
    bool Empty();
    bool Full();
    size_t Size();

    void Clear();

    T& Front();
    T& Back();

    void Enqueue(T value);
    T Dequeue();
};

//constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity)
    : q_array(new T[capacity]), q_capacity(capacity), q_size(0),
```

```
    q_front(0), q_back(-1) {
}

//copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other) {
   q_capacity = other.q_capacity;
   q_size = other.q_size;
   q_front = other.q_front;
   q_back = other.q_back;

   q_array = new T[q_capacity];
   for (size_t i = 0; i < q_capacity; i++) {
       q_array[i] = other.q_array[i];
   }
}

//copy assignment operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
   if (this == &other) {
       return *this;
   }

   T* new_array = new T[other.q_capacity];
   for (size_t i = 0; i < other.q_capacity; ++i) {
       new_array[i] = other.q_array[i];
   }

   delete[] q_array;
   q_array = new_array;
   q_capacity = other.q_capacity;
   q_size = other.q_size;
   q_front = other.q_front;
   q_back = other.q_back;

   return *this;
}

//destructor
template <typename T>
queueArr<T>::~queueArr() {
   delete[] q_array;
}

//empty()
template <typename T>
bool queueArr<T>::Empty() {
   return q_size == 0;
}

//full()
template <typename T>
bool queueArr<T>::Full() {
   return q_size == q_capacity;
}

//size_T
template <typename T>
size_t queueArr<T>::Size() {
   return q_size;
}

// front:
template <typename T>
T& queueArr<T>::Front() {
   //check if the queue is empty
   if (Empty()) {
       std::cout << "Queue is empty" << std::endl;
       return *q_array;
   }
```

```cpp
    //return the front
    return q_array[q_front];
}

// back:
template <typename T>
T& queueArr<T>::Back() {
    //check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is empty" << std::endl;
        return *q_array;
    }
    //return the back
    return q_array[q_back];
}

//clear
template <typename T>
void queueArr<T>::Clear() {
    //reset the q_size, q_front, q_back
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

template <typename T>
void queueArr<T>::Enqueue(T value) {

    //check if the queue is empty
    if (Full()) {
        std::cout << "Queue is full" << std::endl;
        return;
    }

    //move q_back circularly
    q_back = (q_back + 1) % static_cast<int>(q_capacity);

    //store the value to the back
    q_array[q_back] = value;

    //increment the q_size
    ++q_size;
}

//T dequeue();
template <typename T>
T queueArr<T>::Dequeue() {
    // check if empty
    if (Empty()) {
        std::cout << "Queue is empty" << std::endl;
        return T();
    }

    //create temp variable to store front
    T temp = q_array[q_front];

    //move q_front circularly
    q_front = (q_front + 1) % static_cast<int>(q_capacity);

    //decrement the q_size
    --q_size;

    //reset the indexes if queue becomes empty
    if (Empty()) {
        q_front = 0;
        q_back = -1;
    }

    //return the temp var
    return temp;
}
```

```
#endif //QUEUEARR_H
```

OUTPUT



OBSERVATION/REMARKS

Implementation of Linked Lists and Array can be seen and has improved how we can manipulate nodes and data within the nodes to our liking. The completed queue implementation relies on double pointers (qNode<T>**) to track the head and tail of a linked list while. Our manual pointer manipulation directly manages dynamic memory allocations made by the new nodes pointed towards the back and front of our queue to manipulate the data.

SUPPLEMENTARY TASK

MAIN

```
#include <iostream>
#include <string>
#include "jobPrinter.h"
#include "printer.h"

int main() {

   Printer printer(10);
   int numPrint = 0;

   std::cout << "How many jobs do you want to print? ";
   std::cin >> numPrint;

   int next_id = 1;

   for (int i = 0; i < numPrint; i++) {

       std::string name;
       int pages;

       std::cout << "\nEnter your name: ";
       std::cin >> name;

       std::cout << "Enter the number of pages: ";
```

```
        std::cin >> pages;

        printer.aJob(Job(next_id, name, pages));
        next_id++;
    }

    int incomingPrint = 0;

    std::cout << "\nHow many more jobs will come in while the printer runs? ";
    std::cin >> incomingPrint;

     for (int round = 0; round < incomingPrint; round++) {

        std::cout << "\n New Incoming Request " << std::endl;

        std::string name;
        int pages;

        std::cout << "Enter your name: ";
        std::cin >> name;

        std::cout << "Enter number of pages: ";
        std::cin >> pages;

        printer.aJob(Job(next_id, name, pages));
        next_id++;

        if (!printer.Empty()) {
            std::cout << "\nPrinter is processing a job" << std::endl;
            printer.pJob();
        }
    }

    std::cout << "\n Printing Remaining Queue " << std::endl;

    while (!printer.Empty()) {
        printer.pJob();
    }

    std::cout << "\nNo more incoming jobs and the queue is empty."
              << std::endl;

    std::cout << "Shutting down." << std::endl;

    return 0;
}
```

Printer.h HEADER FILE

```
#ifndef PRINTER_QUEUE_H
#define PRINTER_QUEUE_H
#include <iostream>
#include <stdexcept>
#include "jobPrinter.h"

template <typename T>
class queueArr {
private:
    T* q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:
    queueArr(size_t capacity = 10);
    queueArr(const queueArr& other);
    queueArr& operator=(const queueArr& other);
    ~queueArr();

    bool Empty() const;
```

```
    bool Full() const;
    size_t Size() const;
    void Clear();

    T& Front();
    T& Back();

    void Enqueue(const T& value);
    T Dequeue();
};


// Constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity) : q_array(nullptr),
     q_capacity(capacity),
     q_size(0),
     q_front(0),
     q_back(-1) {

    if (capacity > 0) {
        q_array = new T[capacity];
    }
}

// Copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other) : q_array(nullptr),
     q_capacity(other.q_capacity),
     q_size(other.q_size),
     q_front(other.q_front),
     q_back(other.q_back) {

    if (q_capacity > 0) {
        q_array = new T[q_capacity];

        for (size_t i = 0; i < q_capacity; i++) {
            q_array[i] = other.q_array[i];
        }
    }
}

// Assignment operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other) {

    if (this == &other) {
        return *this;
    }

    T* new_array = nullptr;

    if (other.q_capacity > 0) {
        new_array = new T[other.q_capacity];

        for (size_t i = 0; i < other.q_capacity; i++) {
            new_array[i] = other.q_array[i];
        }
    }

    delete[] q_array;

    q_array = new_array;
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    return *this;
}
```

```cpp
// Destructor
template <typename T>
queueArr<T>::~queueArr() {
   delete[] q_array;
}

// Empty
template <typename T>
bool queueArr<T>::Empty() const {
   return q_size == 0;
}

// Full
template <typename T>
bool queueArr<T>::Full() const {
   return q_size == q_capacity;
}

// Size
template <typename T>
size_t queueArr<T>::Size() const {
   return q_size;
}

// Front
template <typename T>
T& queueArr<T>::Front() {

   if (Empty()) {
       throw std::runtime_error("Queue is empty");
   }

   return q_array[q_front];
}

// Back
template <typename T>
T& queueArr<T>::Back() {
   if (Empty()) {
       throw std::runtime_error("Queue is empty");
   }

   return q_array[q_back];
}

// Clear
template <typename T>
void queueArr<T>::Clear() {
   q_size = 0;
   q_front = 0;
   q_back = -1;
}

// Enqueue
template <typename T>
void queueArr<T>::Enqueue(const T& value) {

   if (Full()) {
       std::cout << "Queue is full." << std::endl;
       return;
   }

   q_back = (q_back + 1) % q_capacity;
   q_array[q_back] = value;

   ++q_size;
}

// Dequeue
template <typename T>
T queueArr<T>::Dequeue() {
```

```
    if (Empty()) {
        throw std::runtime_error("Queue is empty");
    }

    T temp = q_array[q_front];

    q_front = (q_front + 1) % q_capacity;
    --q_size;

    if (Empty()) {
        q_front = 0;
        q_back = -1;
    }

    return temp;
}

class Printer {
private:
    queueArr<Job> job_queue;

public:

    Printer(size_t capacity = 10)
        : job_queue(capacity) {
    }

    // Add a job to the printer queue
    void aJob(const Job& job) {

        if (job_queue.Full()) {
            std::cout << "Cannot add Job #"
                      << job.getId()
                      << " - printer queue is full."
                      << std::endl;
            return;
        }

        std::cout << "Submitted print: ";

        job.print();

        std::cout << std::endl;

        job_queue.Enqueue(job);
    }

    // Process all jobs
    void pJob() {

        while (!job_queue.Empty()) {

            Job current = job_queue.Dequeue();

            std::cout << "Printing ";

            current.print();

            std::cout << std::endl;
        }
    }

    // Check if queue is empty
    bool Empty() const {
        return job_queue.Empty();
    }

    // Get queue size
    size_t Size() const {
        return job_queue.Size();
```

```
    }
};

#endif // PRINTER_QUEUE_H
```

jobPrinter.h HEADER FILE

```
#ifndef HOA5SP_JOBPRINTER_H
#define HOA5SP_JOBPRINTER_H
#include <iostream>
#include <string>

class Job {
private:
    int job_id;
    std::string username;
    int numpages;

public:
    Job(int id = 0, std::string user = "", int pages = 0);

    int getId() const;
    std::string getUser() const;
    int getPages() const;

    void print() const;
};

Job::Job(int id, std::string user, int pages)
    : job_id(id),
      username(user),
      numpages(pages) {
}

int Job::getId() const {
    return job_id;
}

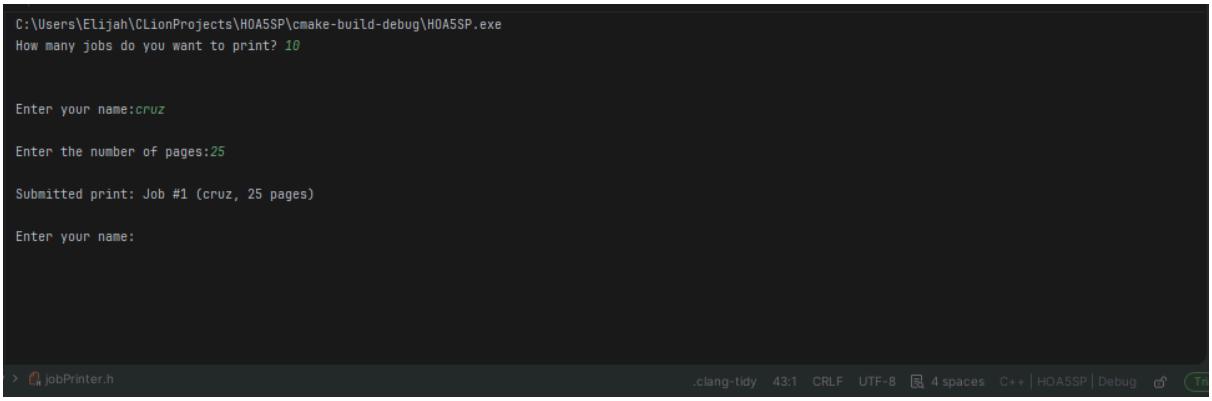
std::string Job::getUser() const {
    return username;
}

int Job::getPages() const {
    return numpages;
}

void Job::print() const {
    std::cout << "Job #" << job_id << " (" << username << ", " << numpages << " pages)";
}

#endif // HOA5SP_JOBPRINTER_H
```

OUTPUT



OUTPUT ANALYSIS

- I used Array in implementing my queue for the program as it is the recently discussed topic that focuses on circular queue arrays that fits the scenario and supplementary task.
- While the code doesn't run perfectly to be honest, it still is able to push requests into the queue. With the reason being as to why the output is the way it is, that is because I still have difficulty in implementing and applying the newly discussed concepts and require more practice.

CONCLUSIONS

As demonstrated during the lecture, the code implements the First-In, First-Out (FIFO) principle through a Queue data structure built with a singly linked list, utilizing standard queue operations like enqueue and dequeue. The program executes these functions by displaying three given letters and then iteratively removing each character until the queue is empty. Integrating both linked list and array concepts enhances our ability to manipulate nodes and internal data effectively. Technically, the queue implementation relies on double pointers (qNode<T>**) to keep track of the head and tail, allowing manual pointer manipulation to directly manage dynamic memory allocations at both ends of the queue. Furthermore, the supplementary material illustrates how to apply these concepts when building programs with queues and lists, reinforcing the fundamental principles and practical purpose of these data structures in C++ by introducing circular array queues and how they can be used and applied to the given scenario or problem.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **GEMINI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of

concepts, etc.

Extent of Use: **Explanation of concepts and how the code runs.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.